

Automated Smart Reminders Engine

companion_pet module — pet_reminders

PawGuard Platform | pawguard-backend (FastAPI + PostgreSQL + Redis + ARQ) | Prepared for Team Review

1. Purpose

Automatically reminds owners about upcoming vaccinations and medications for each pet, and reliably delivers each reminder exactly once through in-app notification + email.

2. Data Model

Table	Key Columns	Purpose
pet_reminders	pet_id, owner_id, kind (vaccination medication), title, details, due_at, source_key (unique), is_active	One row per scheduled reminder
pet_reminder_deliveries	reminder_id, user_id, scheduled_for, delivered_at	One row per actual delivery — unique on (reminder_id, user_id, scheduled_for)

The unique constraint on pet_reminder_deliveries is what guarantees a reminder can never be sent twice for the same scheduled slot, even if the background job re-runs.

3. Endpoints

Method & Path	Permission	Purpose
POST /companion-pets/{pet_id}/reminders	companion_pet:update	Create a reminder for a pet
GET /companion-pets/{pet_id}/reminders	companion_pet:read	List a pet's reminders
DELETE /companion-pets/{pet_id}/reminders/{reminder_id}	companion_pet:update	Cancel a reminder

4. Delivery Pipeline (Scheduled Job)



- send_companion_pet_reminders runs on a daily ARQ cron job at 09:45 (arq_worker.py).
- It fetches every active reminder with due_at within the next 14 days (repository.list_due_reminders).
- For each reminder, deliver_reminder_once() checks the pet_reminder_deliveries table — if that exact (reminder, user, scheduled slot) was already delivered, it's skipped (idempotent).
- Otherwise it delivers via the in-app notification service and email, then writes the delivery record.

5. Covers Both Feature Requests

- Vaccination schedules — kind = “vaccination” (e.g. rabies booster due, annual shots).
- Medication push notifications — kind = “medication” (e.g. daily heartworm pill, post-surgery course).

Both share the exact same table, endpoints, and delivery pipeline — they differ only by the kind field, so no separate subsystem is needed for the two use cases.

Source: companion_pet/models.py (PetReminder, ReminderDelivery, ReminderKind), workers/jobs/companion_pet_jobs.py, workers/arq_worker.py.